# Wprowadzenie do programowania w języku Python

**Sesja 2: Funkcje, dane tekstowe, programowanie obiektowe**

dr inż. Leszek Tarkowski

## Plan sesji

1. Funkcje, moduły, wyjątki
2. Przetwarzanie tekstu i plików
3. Programowanie obiektowe

Dziś po raz pierwszy wczytamy cały zbiór: 1309 pasażerów Titanica z pliku `titanic3.csv`.

Jeśli blok 1.6 (pętle) nie zmieścił się w sesji 1, zaczynamy od niego: slajdy są na końcu sesja1.qmd.

## Przypomnienie

```
passengers = [
    {"name": "Allen, Miss. Elisabeth Walton",
     "pclass": 1, "age": 29.0},
    {"name": "Allison, Master. Hudson Trevor",
     "pclass": 1, "age": 0.92},
]

for p in passengers:
    if p["age"] is not None and p["age"] < 18:
        print(p["name"], "jest dzieckiem")
```

- Pasażer to słownik, lista słowników to tabela.
- Pętla przechodzi po wierszach, warunek wybiera te, które nas interesują.

## 2.1 Funkcje, moduły, wyjątki

### Po co funkcje?

Ten sam fragment w trzech miejscach programu:

```
if age is None:
    group = "nieznany"
elif age < 18:
    group = "dziecko"
...
```

- Zmiana progu (18 → 16) wymaga poprawek w trzech miejscach.
- Łatwo o pomyłkę: jedno miejsce poprawione, dwa nie.
- Kod nie mówi, *co* robi, tylko *jak*.

Funkcja to **nazwany fragment kodu**: piszemy go raz, używamy wiele razy.

### Definicja funkcji

```
1  def age_group(age):
2      if age is None:
3          return "nieznany"
4      if age < 18:
5          return "dziecko"
6      if age < 60:
7          return "dorosły"
8      return "senior"
9
10 age_group(29)           # 'dorosły'
11 age_group(None)         # 'nieznany'
```

- `def`, nazwa, **parametry** w nawiasach, dwukropek, wcięty blok.
- `return` kończy funkcję i oddaje wynik.

### `return` a `print`

```
def greet(first_name):
    print(f"Witaj, {first_name}!")

result = greet("Ala")       # wypisuje: Witaj, Ala!
print(result)               # None
```

- `print` **pokazuje** wartość na ekranie; programowi nic nie zostaje.
- `return` **oddaje** wartość; można ją zapisać, porównać, przekazać dalej.
- Funkcja bez `return` zwraca `None`.

Funkcje obliczające coś powinny zwracać wynik, a nie go wypisywać.

### Parametry pozycyjne i nazwane

```
def ticket_category(fare, cheap_below=10, expensive_from=50):
    if fare < cheap_below:
        return "tani"
    if fare < expensive_from:
        return "średni"
    return "drogi"

ticket_category(26.55)                    # 'średni'
ticket_category(26.55, expensive_from=20)  # 'drogi'
ticket_category(fare=8.05)                # 'tani'
```

- Parametry z **wartością domyślną** można pominąć.
- Argumenty **nazwane** (`expensive_from=20`) można podać w dowolnej kolejności.

### Pułapka: zmienna wartość domyślna

```
def add(element, items=[]):
    items.append(element)
    return items

add(1)        # [1]
add(2)        # [1, 2]  ← ta sama lista!
```

Wartość domyślna powstaje **raz**, przy definicji funkcji. Poprawnie:

```python
def add(element, items=None):
    if items is None:
        items = []
    items.append(element)
    return items
```

To ta sama pułapka co aliasowanie z sesji 1: jedna lista, wiele odwołań.

### Zasięg zmiennych

```python
vat_rate = 0.23              # zmienna globalna

def gross(net):
    vat = net * vat_rate # zmienna lokalna
    return net + vat

gross(100)                   # 123.0
print(vat)                   # NameError: name 'vat' is not defined
```

- Zmienne utworzone w funkcji istnieją tylko w niej.
- Funkcja może **czytać** zmienne globalne, ale lepiej przekazywać dane parametrami.

### Dokumentacja funkcji

```python
def average(numbers: list[float]) -> float | None:
    """Zwraca średnią arytmetyczną albo None dla pustej listy."""
    if not numbers:
        return None
    return sum(numbers) / len(numbers)

help(average)
```

- **Docstring**: opis w potrójnym cudzysłowie, widoczny w `help()` i w podpowiedziach VS Code.
- **Adnotacje typów** (`: list[float]`, `-> float | None`): dokumentacja dla ludzi i narzędzi; Python ich nie sprawdza.

### Funkcja jako narzędzie dekompozycji

Duży problem dzielimy na kroki, a każdy krok to funkcja:

```
def main():
    rows = load("data/titanic3.csv")
    passengers = clean(rows)
    results = survival_by_class(passengers)
    print_report(results)
```

- Każdą funkcję można napisać, przeczytać i sprawdzić osobno.
- `main()` czyta się jak spis treści programu.

### Funkcja jako argument

```
names = ["Allen", "Cardeza", "Ali", "Andersson"]

sorted(names)              # ['Ali', 'Allen', 'Andersson', 'Cardeza']
sorted(names, key=len)     # ['Ali', 'Allen', 'Cardeza', 'Andersson']
max(names, key=len)        # 'Andersson'
```

`key=` przyjmuje **funkcję**, która mówi, według czego porównywać.

```
sorted(passengers, key=lambda p: p["age"])
```

`lambda` to krótka funkcja bez nazwy, zapisana w jednej linii.

### Pokaz w VS Code

- Funkcje `age_group()` i `ticket_category()` z docstringiem.
- Porównanie `print` i `return` w REPL.
- Pułapka zmiennej wartości domyślnej i jej poprawka.
- `help()` na własnej funkcji, podpowiedź w VS Code po najechaniu myszą.
- Sortowanie listy pasażerów według wieku przez `key=`.

### Moduły i `import`

```python
import math
math.sqrt(2)                  # 1.4142135623730951

from math import pi, floor
floor(2.7)                    # 2

import statistics as st
st.median([22, 29, 30, 47])   # 29.5
```

- **Moduł** to plik `.py` z funkcjami, klasami i stałymi.
- Własny moduł: plik `tools.py` w tym samym folderze, potem `import tools`.

### `if __name__ == "__main__"`

```python
# tools.py
def age_group(age):
    ...

if __name__ == "__main__":
    print(age_group(29))          # test przy uruchomieniu pliku
```

- `python tools.py`: blok się wykonuje.
- `import tools`: blok jest pomijany, dostępne są tylko funkcje.

Ten sam plik może być programem i biblioteką.

### Biblioteka standardowa

| Moduł | Do czego |
|---|---|
| `math` | funkcje matematyczne |
| `random` | liczby losowe, losowanie z listy |
| `statistics` | średnia, mediana, odchylenie standardowe |
| `datetime` | daty i czas |
| `csv`, `json` | popularne formaty plików |
| `pathlib` | ścieżki do plików |
| `collections` | `Counter` i inne kolekcje |
| `re` | wyrażenia regularne |

Wszystko jest dostępne od razu po instalacji Pythona.

### `random`, `statistics`, `datetime`

```
import random, statistics
from datetime import date

random.seed(42)                         # powtarzalne losowanie
random.choice(["S", "C", "Q"])

statistics.mean([22, 29, 30, 47])       # 32
statistics.stdev([22, 29, 30, 47])      # 10.61...

sinking = date(1912, 4, 15)
sinking.strftime("%d.%m.%Y")            # '15.04.1912'
(date(2026, 11, 7) - sinking).days # 41844
```

## Wyjątki: przypomnienie

```
text = input("Podaj wiek: ")
try:
    age = int(text)
except ValueError:
    print(f"„{text}" to nie jest liczba całkowita.")
    age = None
```

- Kod w `try` wykonuje się normalnie.
- Gdy pojawi się wskazany wyjątek, wykonuje się `except`, a program działa dalej.

Dziś: pełna postać instrukcji `try` i zgłaszanie własnych wyjątków.

## Pełna postać `try`

```
1 def divide(a, b):
2     try:
3         result = a / b
4     except ZeroDivisionError:
5         print("Dzielenie przez zero")
6         return None
7     except TypeError as e:
```

```
        print("Zły typ:", e)
        return None
    else:
        return result
    finally:
        print("koniec podziel()")
```

`else` wykonuje się, gdy wyjątku **nie było**; `finally` wykonuje się **zawsze**.

`as e` daje dostęp do obiektu wyjątku; `print(e)` wypisuje jego opis. finally typowo zamyka zasoby, ale przy plikach robi to za nas `with`.

### Zgłaszanie wyjątku: `raise`

```
def read_age(text):
    if text.strip() == "":
        return None
    age = float(text.replace(",", "."))
    if not 0 <= age <= 120:
        raise ValueError(f"wiek poza zakresem: {age}")
    return age
```

| Wejście | Wynik |
|---|---|
| `"29"`, `"0,92"` | `29.0`, `0.92` |
| `""` | `None` (brak danych) |
| `"abc"` | `ValueError: could not convert string to float: 'abc'` |
| `"-5"` | `ValueError: wiek poza zakresem: -5.0` |

### Rodziny wyjątków

```
Exception
    ArithmeticError
        ZeroDivisionError
    LookupError
        KeyError
        IndexError
    ValueError
    TypeError
```

- `except LookupError:` łapie i `KeyError`, i `IndexError`.
- Bloki `except` sprawdzane są po kolei: **szczegółowe przed ogólnymi**.
- `except Exception:` łapie prawie wszystko, więc używamy go rzadko i świadomie.

### Pokaz w VS Code

- Własny moduł `tools.py` z funkcjami z tej części; import w drugim pliku.
- `if __name__ == "__main__"` jako miejsce na szybkie testy.
- Funkcja `read_age()` z `raise`; sprawdzenie dla kilku wejść.
- `try` / `except` / `else` / `finally`: kolejność wypisywanych komunikatów.

## 2.2 Przetwarzanie tekstu i plików

### Metody `str`

```
s = "  Allen, Miss. Elisabeth Walton  "

s.strip()                    # 'Allen, Miss. Elisabeth Walton'
s.strip().lower()            # 'allen, miss. elisabeth walton'
s.strip().split(", ")        # ['Allen', 'Miss. Elisabeth Walton']
"; ".join(["S", "C", "Q"])   # 'S; C; Q'
s.replace("Miss.", "Panna")  # '  Allen, Panna Elisabeth Walton  '
s.strip().startswith("All")  # True
s.find("Miss")               # 9
s.count("l")                 # 4
```

Każda metoda **zwraca nowy napis**: `str` jest niezmienny, oryginał zostaje bez zmian.

### Rozbiór nazwiska

Pole `name` ma stały układ: `Nazwisko, Tytuł. Imiona`

```
1 full_name = "Allen, Miss. Elisabeth Walton"
2
3 surname, _, rest = full_name.partition(", ")
4 title, _, first_names = rest.partition(". ")
5
6 surname, title, first_names  # ('Allen', 'Miss', 'Elisabeth Walton')
```

- `partition()` dzieli na trzy części: przed separatorem, separator, po nim.
- `_` to umowna nazwa zmiennej, której wartość nas nie interesuje.

### Tekst a bajty: kodowanie znaków

Plik na dysku to ciąg **bajtów**. Napis `str` to ciąg **znaków** Unicode.

```
"ł".encode("utf-8")              # b'\xc5\x82'  (2 bajty)
"ł".encode("cp1250")             # b'\xb3'      (1 bajt)

"ł".encode("utf-8").decode("cp1250")   # 'Ĺ¦'  ← „krzaczki"
b"\xb3".decode("utf-8")                # UnicodeDecodeError
```

| Kodowanie | Gdzie spotkamy |
|---|---|
| UTF-8 | standard: Python, WWW, macOS, Linux |
| cp1250 | starsze polskie programy i pliki z Windows |
| UTF-8 z BOM (`utf-8-sig`) | CSV zapisany w Excelu jako „UTF-8” |

### Otwieranie pliku: `with open`

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Pierwsza linia\n")
    f.write("Zażółć gęślą jaźń\n")

with open("notes.txt", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip("\n"))
```

- Tryby: `"r"` odczyt (domyślny), `"w"` zapis od nowa, `"a"` dopisywanie.
- `with` zamyka plik automatycznie, także po błędzie.
- `encoding=` podajemy **zawsze**: domyślne kodowanie zależy od systemu.

### Ścieżki: `pathlib`

```python
from pathlib import Path

path = Path("data") / "titanic3.csv"
path.exists()                # True
path.suffix                  # '.csv'
path.stat().st_size          # 116752 (bajtów)

for file in Path("data").glob("*.csv"):
    print(file.name)
```

Ścieżka względna liczona jest od **bieżącego katalogu**, nie od pliku ze skryptem. W VS Code otwieramy cały folder projektu.

### CSV: moduł `csv`

```python
import csv

with open("data/titanic3.csv", encoding="utf-8", newline="") as f:
    reader = csv.DictReader(f)
    rows = list(reader)

len(rows)                # 1309
reader.fieldnames[:4]    # ['pclass', 'survived', 'name', 'sex']
```

- `DictReader`: każdy wiersz to słownik z kluczami z nagłówka.
- Moduł `csv` poprawnie obsługuje cudzysłowy i przecinki wewnątrz pól, np. `"St Louis, MO"`.

### Wszystko jest tekstem

```python
rows[0]
```

```
{'pclass': '1', 'survived': '1',
 'name': 'Allen, Miss. Elisabeth Walton',
 'sex': 'female', 'age': '29', 'sibsp': '0', 'parch': '0',
 'ticket': '24160', 'fare': '211.3375', 'cabin': 'B5',
 'embarked': 'S', 'boat': '2', 'body': '', 'home.dest': 'St Louis, MO'}
```

- Liczby są napisami: `'1'`, `'29'`, `'211.3375'`.
- Brak danych to pusty napis `''`.
- Konwersję typów i braki musimy obsłużyć sami.

### Ile danych brakuje?

```
for column in ["age", "fare", "cabin", "embarked"]:
    missing = sum(1 for w in rows if w[column] == "")
    print(f"{column:<9} {missing:>5}  ({missing / len(rows):.0%})")
```

```
age          263  (20%)
fare           1  (0%)
cabin       1014  (77%)
embarked       2  (0%)
```

```
def number_or_none(text):
    return float(text) if text.strip() else None
```

### CSV z polskiego Excela

```
with open("sprzedaz.csv", encoding="utf-8-sig", newline="") as f:
    for row in csv.DictReader(f, delimiter=";"):
        amount = float(row["kwota"].replace(",", "."))
```

| Problem | Rozwiązanie |
|---|---|
| separator pól to średnik | `delimiter=";"` |
| przecinek dziesiętny | `.replace(",", ".")` przed `float()` |
| znak BOM na początku pliku | `encoding="utf-8-sig"` |
| starszy Excel, krzaczki | `encoding="cp1250"` |

### Zapis CSV

```python
results = [
    {"pclass": 1, "passengers": 323, "survivors": 200},
    {"pclass": 2, "passengers": 277, "survivors": 119},
    {"pclass": 3, "passengers": 709, "survivors": 181},
]

with open("results.csv", "w", encoding="utf-8", newline="") as f:
    fields = ["pclass", "passengers", "survivors"]
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerows(results)
```

`newline=""` przy plikach CSV: moduł `csv` sam dba o końce linii.

### JSON

```python
import json

passenger = {"name": "Allen", "age": 29.0,
             "cabin": None, "survived": True}
json.dumps(passenger)
# '{"nazwisko": "Allen", "wiek": 29.0, "kabina": null, "przezyl": true}'

with open("pasazer.json", "w", encoding="utf-8") as f:
    json.dump(passenger, f, indent=2)
with open("pasazer.json", encoding="utf-8") as f:
    loaded = json.load(f)
```

JSON to format wymiany danych w WWW i API; odpowiada słownikom i listom Pythona.

Przy polskich znakach dodajemy ensure_ascii=False, inaczej „ł” zostanie zapisane jako 0142.

### Wyrażenia regularne

Wzorce opisujące **kształt** tekstu:

| Wzorzec | Znaczenie |
|---|---|
| `\d` | cyfra |
| `\s` | biały znak (spacja, tabulator) |

| Wzorzec | Znaczenie |
|---|---|
| `[^.]` | dowolny znak oprócz kropki |
| `+` | co najmniej jeden raz |
| `( … )` | grupa: fragment do wyłuskania |

```
import re

re.fullmatch(r"\d+", "24160")          # pasuje: bilet z samych cyfr
re.fullmatch(r"\d+", "PC 17609")       # None
re.findall(r"\d+", "C22 C26")          # ['22', '26']
```

### Tytuł z nazwiska

```
import re

def title(name):
    found = re.search(r",\s*([^.]+)\.", name)
    return found.group(1) if found else None

title("Allen, Miss. Elisabeth Walton")      # 'Miss'
title("Allison, Master. Hudson Trevor")     # 'Master'
```

Wzorzec: przecinek, odstęp, **grupa znaków innych niż kropka**, kropka.

Prostszą alternatywą jest partition(), ale wyrażenie regularne lepiej znosi nietypowe zapisy. Na całym zbiorze funkcja znajduje tytuł we wszystkich 1309 wierszach.

### Ile jakich tytułów?

```
from collections import Counter

titles = Counter(title(w["name"]) for w in rows)
titles.most_common(6)
```

```
[('Mr', 757), ('Miss', 260), ('Mrs', 197),
 ('Master', 61), ('Dr', 8), ('Rev', 8)]
```

- `Counter` zlicza wystąpienia, czyli wzorzec „słownik zliczający” z sesji 1 w jednej linii.
- *Master* to tytuł chłopców; w 1912 r. w ten sposób zapisywano dzieci.

### Pokaz w VS Code

- Wczytanie `titanic3.csv` przez `csv.DictReader`; podgląd pierwszego wiersza.
- Liczba braków w kolumnach `age`, `cabin`, `embarked`.
- Funkcja `title()` i `Counter` tytułów.
- Zapis wyników do `results.csv` i otwarcie pliku w Excelu.
- Zapis tekstu z polskimi znakami w `cp1250`, odczyt jako UTF-8: `UnicodeDecodeError`.

## 2.3 Programowanie obiektowe

### Od słownika do klasy

```
p = {"name": "Allen, Miss. Elisabeth Walton",
     "pclass": 1, "age": 29.0}

p["age"] = -5               # nikt tego nie sprawdzi
p["aeg"] = 30               # literówka: powstał nowy klucz
age_group(p["age"])         # funkcja żyje osobno od danych
```

Słownik nie pilnuje:

- jakie pola ma mieć pasażer,
- jakie wartości są poprawne,
- jakie operacje mają sens dla tych danych.

**Klasa** łączy dane i zachowanie w jeden typ.

### Wszystko jest obiektem

Od pierwszego wykładu korzystamy z obiektów:

```
text = "titanic"
text.upper()           # metoda obiektu typu str
items = [3, 1, 2]
items.sort()           # metoda obiektu typu list
type(items)            # <class 'list'>
```

- **Obiekt** ma stan (dane) i zachowanie (metody).
- **Klasa** to typ obiektu: przepis, według którego obiekty powstają.
- `str`, `list`, `dict` to klasy wbudowane. Dziś napiszemy własną.

### Pierwsza klasa

```
class Passenger:
    def __init__(self, name, pclass, age=None):
        self.name = name
        self.pclass = pclass
        self.age = age

    def age_group(self):
        if self.age is None:
            return "nieznany"
        return "dziecko" if self.age < 18 else "dorosły"

p = Passenger("Allen, Miss. Elisabeth Walton", 1, 29.0)
q = Passenger("Allison, Master. Hudson Trevor", 1, 0.92)
```

### Klasa, instancja, `self`

```
p.name               # 'Allen, Miss. Elisabeth Walton'
q.age_group()        # 'dziecko'
type(p)              # <class '__main__.Passenger'>
```

- **Klasa** `Passenger`: jedna definicja. **Instancje** `p` i `q`: konkretni pasażerowie.
- `__init__` przygotowuje nowy obiekt; wywołuje się przy `Passenger(...)`.
- `self` to obiekt, na którym działa metoda.

```
q.age_group()                  # zapis skrócony
Passenger.age_group(q)         # to samo: self = q
```

### Jak obiekt się przedstawia

```
print(p)    # <__main__.Passenger object at 0x10a3f2b50>
```

```
class Passenger:
    ...
    def __str__(self):
        return f"{self.name} (klasa {self.pclass})"
```

```
    def __repr__(self):
        return f"Passenger({self.name!r}, {self.pclass}, {self.age})"
```

```
print(p)    # Allen, Miss. Elisabeth Walton (klasa 1)
[p, q]      # [Passenger('Allen, Miss. Elisabeth Walton', 1, 29.0), ...]
```

- `__str__`: dla użytkownika (`print`, f-string). `__repr__`: dla programisty (REPL, listy).

### Metody specjalne

Metody z podwójnym podkreśleniem wywołuje sam Python:

| Zapis | Wywołuje |
|---|---|
| `Passenger(...)` | `__init__` |
| `print(p)`, `f"{p}"` | `__str__` |
| `p == q` | `__eq__` |
| `len(x)` | `__len__` |
| `for e in x` | `__iter__` |
| `a in x` | `__contains__` |

```
def __eq__(self, other):
    return self.name == other.name and self.pclass == other.pclass
```

Bez `__eq__` porównanie `==` sprawdza tożsamość, czyli działa jak `is`.

### Pokaz w VS Code

- Klasa `Passenger` z `__init__` i metodą `age_group()`.
- Kilka instancji, wywołanie metody na każdej.
- `print(p)` przed i po dodaniu `__str__`; lista pasażerów przed i po `__repr__`.
- `p == q` przed i po dodaniu `__eq__`.

### Hermetyzacja

```
p.age = -5          # klasa nadal na to pozwala
```

- W Pythonie nie ma pól „prywatnych”: obowiązuje **konwencja**.

- Nazwa z podkreśleniem `_age` oznacza: szczegół wewnętrzny, nie ruszać z zewnątrz.
- Dostęp do danych kontrolujemy przez `@property`.

### `@property`: walidacja przy zapisie

```
class Passenger:
    def __init__(self, age=None):
        self.age = age           # przez setter

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, new):
        if new is not None and new < 0:
            raise ValueError(f"ujemny wiek: {new}")
        self._age = new
```

Teraz p.age = -5 zgłasza ValueError, a zapis p.age nadal wygląda jak zwykłe pole. Walidacja działa też w **init**, bo self.age = age przechodzi przez setter.

### `@dataclass`

Klasy przechowujące dane mają powtarzalny kod. `@dataclass` pisze go za nas:

```
from dataclasses import dataclass

@dataclass
class Passenger:
    name: str
    pclass: int
    sex: str
    survived: bool
    age: float | None = None
    relatives: int = 0
    fare: float | None = None
```

Automatycznie powstają `__init__`, `__repr__` i `__eq__`.

### `@dataclass` w działaniu

```
p = Passenger("Allen, Miss. Elisabeth Walton", 1, "female", True,
              age=29.0, fare=211.3375)
p
```

```
Passenger(name='Allen, Miss. Elisabeth Walton', pclass=1, sex='female',
          survived=True, age=29.0, relatives=0, fare=211.3375)
```

- Pola z typami czytamy jak opis rekordu: od razu widać, co ma pasażer.
- Pola z wartością domyślną muszą być na końcu.
- Metody dopisujemy jak w zwykłej klasie.

### Walidacja: `__post_init__`

```
@dataclass
class Passenger:
    ...

    def __post_init__(self):
        if self.pclass not in (1, 2, 3):
            raise ValueError(f"nieznana klasa: {self.pclass}")
        if self.age is not None and self.age < 0:
            raise ValueError(f"ujemny wiek: {self.age}")
```

```
Passenger("X", 4, "male", False)  # ValueError: nieznana klasa: 4
```

`__post_init__` wywołuje się zaraz po automatycznym `__init__`: obiekt z błędnymi danymi w ogóle nie powstaje.

### Alternatywny konstruktor: `@classmethod`

```
    @classmethod
    def from_csv_row(cls, row: dict) -> "Passenger":
        return cls(
            name=row["name"],
            pclass=int(row["pclass"]),
```

```
            sex=row["sex"],
            survived=row["survived"] == "1",
            age=number_or_none(row["age"]),
            relatives=int(row["sibsp"]) + int(row["parch"]),
            fare=number_or_none(row["fare"]),
        )
```

Metoda klasy dostaje **klasę** (`cls`), a nie instancję. Konwersja z formatu pliku jest w jednym miejscu.

Wywołanie: Passenger.from_csv_row(row). Dla porządku: @staticmethod to zwykła funkcja umieszczona w klasie, bez self i bez cls.

### Kompozycja: lista pasażerów

```
class PassengerList:
    def __init__(self, passengers):
        self._passengers = list(passengers)

    def survival_rate(self):
        if not self._passengers:
            return None
        survivors = sum(p.survived for p in self._passengers)
        return survivors / len(self._passengers)
```

- `PassengerList` **zawiera** obiekty `Passenger`: to jest kompozycja.
- Obliczenia na całej grupie mieszkają w klasie, która tę grupę przechowuje.

### Wczytanie całego pliku

```
class PassengerList:
    ...
    @classmethod
    def from_csv_file(cls, path):
        with open(path, encoding="utf-8", newline="") as f:
            rows = csv.DictReader(f)
            return cls(Passenger.from_csv_row(w) for w in rows)
```

```
passengers = PassengerList.from_csv_file("data/titanic3.csv")
f"{passengers.survival_rate():.1%}"  # '38.2%'
```

Dwa konstruktory klasowe: `Passenger.from_csv_row()` tworzy jednego pasażera, `PassengerList.from_csv_file()` wszystkich.

### Protokoły: obiekt jak wbudowana kolekcja

```
class PassengerList:
    ...
    def __len__(self):
        return len(self._passengers)

    def __iter__(self):
        return iter(self._passengers)

    def __contains__(self, name):
        return any(p.name == name for p in self._passengers)
```

```
passengers = PassengerList.from_csv_file("data/titanic3.csv")
len(passengers)                              # 1309
"Allen, Miss. Elisabeth Walton" in passengers  # True
sum(p.survived for p in passengers)          # 500
```

### Duck typing

> Jeśli chodzi jak kaczka i kwacze jak kaczka, to jest kaczką.

Python nie pyta o typ obiektu, tylko o to, **co obiekt potrafi**.

```
for p in passengers: ...                # działa, bo jest __iter__
max(passengers, key=lambda p: p.fare or 0) # działa z każdą kolekcją
sorted(passengers, key=lambda p: p.name)
```

`PassengerList` współpracuje z `for`, `len`, `in`, `max`, `sorted` i `sum`, choć żadna z tych funkcji nie wie o jej istnieniu.

### Filtrowanie: funkcja jako argument

```
class PassengerList:
    ...
    def filter(self, condition):
        return PassengerList(p for p in self._passengers if condition(p))
```

```
for k in (1, 2, 3):
    group = passengers.filter(lambda p: p.pclass == k)
    print(f"klasa {k}: {group.survival_rate():.1%}")
```

```
klasa 1: 61.9%
klasa 2: 43.0%
klasa 3: 25.5%
```

### Pokaz w VS Code

- `Passenger` jako `@dataclass` z `__post_init__` i `from_csv_row()`.
- `PassengerList.from_csv_file()`: wczytanie 1309 obiektów.
- `len`, `in`, pętla `for` po własnej klasie.
- Przeżywalność według klasy, płci i grupy wiekowej przez `filter()`.
- Najdroższy bilet przez `max(..., key=...)`.

### Dziedziczenie

```
1  class Person:
2      def __init__(self, name):
3          self.name = name
4
5      def describe(self):
6          return self.name
7
8  class Passenger(Person):
9      def __init__(self, name, pclass):
10         super().__init__(name)
11         self.pclass = pclass
```

- `Passenger` **jest** `Person`: dziedziczy pola i metody, w tym `describe()`.
- `super().__init__(...)` uruchamia część wspólną z klasy bazowej.

### Nadpisywanie metod

```
class Passenger(Person):
    ...
    def describe(self):
        return f"{super().describe()}, pasażer klasy {self.pclass}"
```

```
p = Passenger("Allen, Miss. Elisabeth Walton", 1)
p.describe()  # 'Allen, Miss. Elisabeth Walton, pasażer klasy 1'
```

- Metoda zdefiniowana w klasie pochodnej **zastępuje** metodę z klasy bazowej.
- `super().describe()` pozwala skorzystać z wersji bazowej zamiast ją powtarzać.

### Wiele klas pochodnych

```
class CrewMember(Person):
    def __init__(self, name, role):
        super().__init__(name)
        self.role = role

    def describe(self):
        return f"{self.name}, załoga: {self.role}"
```

```
on_board = [Passenger("Allen, Miss. Elisabeth Walton", 1),
            CrewMember("Smith, Edward", "kapitan")]
for person in on_board:
    print(person.describe())
```

```
Allen, Miss. Elisabeth Walton, pasażer klasy 1
Smith, Edward, załoga: kapitan
```

Przykład hipotetyczny: zbiór titanic3 nie zawiera załogi.

### Polimorfizm

Różne klasy, ta sama metoda `save()`, różne zachowanie:

```
class TextReport:
    def save(self, results, file):
        for name, value in results.items():
            file.write(f"{name:<10} {value:6.1%}\n")

class CsvReport:
    def save(self, results, file):
        writer = csv.writer(file)
        writer.writerow(["grupa", "przezywalnosc"])
        for name, value in results.items():
            writer.writerow([name, round(value, 3)])
```

```
def generate(report, results, file):
    report.save(results, file)
```

generate() nie sprawdza typu raportu: wystarczy, że obiekt ma metodę save(). To ten sam duck typing co wcześniej.

### Kompozycja czy dziedziczenie?

| | Kompozycja | Dziedziczenie |
|---|---|---|
| Relacja | **ma** (*has-a*) | **jest** (*is-a*) |
| Przykład | lista pasażerów *ma* pasażerów | pasażer *jest* osobą |
| Powiązanie klas | luźne | ścisłe |
| Zmiana klasy bazowej | nie dotyka innych | dotyka wszystkich pochodnych |

Zasada praktyczna: zaczynamy od kompozycji. Dziedziczenia używamy, gdy relacja „jest” jest naprawdę oczywista.

### Proceduralnie czy obiektowo?

Jaki odsetek dzieci przeżył?

**Proceduralnie**

```
rows = load(file)
children = [w for w in rows if w["age"] and float(w["age"]) < 18]
p = sum(w["survived"] == "1" for w in children) / len(children)
```

**Obiektowo**

```
passengers = PassengerList.from_csv_file(file)
children = passengers.filter(lambda p: p.age_group() == "dziecko")
p = children.survival_rate()
```

Wynik ten sam: **52,6%**. Różnica: gdzie mieszka wiedza o danych.

## Kiedy klasa się opłaca?

**Warto**, gdy:

- dane i operacje na nich tworzą całość (pasażer, faktura, konto),
- te same dane występują w wielu miejscach programu,
- trzeba pilnować poprawności danych (walidacja w jednym miejscu),
- program będzie rozwijany i czytany przez innych.

**Nie warto**, gdy:

- to jednorazowy skrypt na kilkadziesiąt linii,
- wystarczą funkcje i słowniki.

## Pokaz w VS Code

- `Person` → `Passenger`, `CrewMember`; wspólna pętla po `describe()`.
- `TextReport` i `CsvReport`; ta sama funkcja `generate()` dla obu.
- Zapis raportu do pliku i na ekran (`sys.stdout`).
- Ta sama analiza przeżywalności dzieci w wersji proceduralnej i obiektowej.

## Podsumowanie

- Funkcja to nazwany fragment kodu: parametry na wejściu, `return` na wyjściu.
- Moduły porządkują kod; biblioteka standardowa daje gotowe narzędzia.
- Wyjątki obsługujemy (`try`) i zgłaszamy (`raise`) świadomie.
- Pliki otwieramy przez `with`, zawsze z `encoding=`; CSV czytamy modułem `csv`.
- Klasa łączy dane z zachowaniem; `@dataclass` skraca zapis.
- Kompozycja („ma”) przed dziedziczeniem („jest”); polimorfizm to wspólny interfejs.